

Poliscope

技术白皮书

**EpistemoBrain：七人科学议会、双图证据治理与
计算社会科学盲点发现智能体**

版本 1.0 2026 年 8 月

对应系统论文：

《EpistemoBrain: A Seven-Scientist Council with Dual-Graph Evidence Governance
for Auditable Scientific Blindspot Discovery》
(见 [docs/paper/epistemobrain.pdf](#))

目录

1	引言：为什么需要一场没有和稀泥的科研审查	2
1.1	问题：流畅的摘要掩盖了证据的裂缝	2
1.2	对策：不是更快的综述机器，而是一面照出证据裂缝的透镜	2
1.3	三点贡献	3
1.4	本白皮书的组织	3
2	EpistemoBrain：核心方法体系	3
2.1	方法定位：一个组织，不是一个聊天机器人	3
2.2	三根支柱	4
2.3	一个研究任务的认识论循环	4
2.4	认识论立场：三个「不」	4
2.5	与普通 Deep Research / Multi-Agent Debate 的本质区别	5
3	系统架构总览	5
3.1	模块化单体	5
3.2	一次研究的生命周期	6
3.3	为什么拆两个事务、两个身份	6
3.4	为什么队列在 PostgreSQL 而不是 Redis	6
4	七人科学议会	7
4.1	七个席位	7
4.2	八个阶段（PHASE_SEQUENCE）	7
4.3	结构化科研动作	8
4.4	禁止多数投票：条件化共识	8
4.5	人类方向性引导检查点（AWAITING_COUNCIL_INPUT）	9
5	三层记忆与 MemoBrain 适配	9
5.1	记忆不是私有缓存，而是公共认知层	9
5.2	MemoBrain 集成规则	10
5.3	席位私有记忆：一个规则保证隔离	10
5.4	过程记忆的 Fold：保骨架不保字数	10
5.5	Perspective Recall：角色化召回	10
5.6	认识论路由：让记忆图主动调度科学家	11
6	双图架构与事件账本	11
6.1	为什么是两张图	11
6.2	证据图：首版正式节点	11
6.3	证据图：首版正式边	12
6.4	科研事件账本（Scientific Event Ledger）	12

6.5	Graph Projector: 唯一写入者	12
6.6	双图如何双向联动	13
6.7	被反驳、隔离、折叠的节点不物理删除	13
7	证据门、三层审计与证据谱系	13
7.1	六阶段证据门	13
7.2	证据分层 A-D	14
7.3	三层审计	14
7.4	因果越级拦截	14
7.5	证据谱系与独立性	14
7.6	防止共享证据错误的四种机制	14
8	token 与成本控制: 七人全程如何不烧钱	15
8.1	发言预算: 每轮每席有上限	15
8.2	新颖性门控: 先去重再推理	15
8.3	分层模型调用: 复杂判断用强模型, 机械操作轻量模型	15
8.4	共享检索缓存: 七人合查一个池	16
8.5	共享缓存与模型调用的可观测成本	16
8.6	四维预算: 不是只卡一个数	16
8.7	阶段级 deadline: 防止单个席位拖死全体	16
8.8	800 字符的 Recall 预算: 上下文竞争管理	16
8.9	Free Trial 与 BYOK: 成本在研究者手里	17
9	可靠性、状态机与快照	17
9.1	显式状态机	17
9.2	快照 / 暂停 / 恢复 = 「暂停认领」	17
9.3	检查点与断点续跑	17
9.4	单席失败降级	18
9.5	停止即时生效	18
9.6	重试与超时策略	18
9.7	会话删除: 研究者可以销毁自己的会话	18
9.8	测试: 三层测试纪律	18
10	ForesightBlindspot 评测体系	19
10.1	为什么需要专门的时间切片基准	19
10.2	评测目标与五条基线	19
10.3	核心指标族	19
10.4	评测管道与生产系统同构	20
10.5	基线构造正确性: 评测管道发现的修复	20
10.6	当前评测的诚实边界	20
10.7	盲点质量裁判: 关键词代理到语义评判	20

10.8 真实模型运行的结果与边界	21
11 工程实现细节	21
11.1 技术栈	21
11.2 模型网关	22
11.3 工具网关	22
11.4 检索与取证管线	22
11.5 报告组装	23
11.6 SSE 实时流：两条流一份契约	23
11.7 权限即设计	23
11.8 部署	24
11.9 前端设计取向	24
11.10 安全与伦理	24

1 引言：为什么需要一场没有和稀泥的科研审查

1.1 问题：流畅的摘要掩盖了证据的裂缝

大型语言模型（LLM）智能体让深度研究变得触手可及：研究者输入一个问题，智能体检索、阅读、综合出一篇读起来通顺的综述。但对于存在争议的科研问题，标准的深度研究管线继承了三个单靠优化文风无法修复的失效模式。

第一，单一视角。一个智能体读一篇论文，倾向于只看到自己那一路的解释。测量科学家对自报告偏差的担忧，在一个根本没有测量科学家的管线上是不可见的。

第二，一致不是证据。如果七个智能体读到的是同一份错误的摘要、同一篇被撤稿的论文、同一条张冠李戴的引用，系统可以把一个共享的错误包装成一个看起来被反复验证过的共识——因为被重复而被当成被证实。

第三，总结器拍平异议。总结器的最后一步「大家基本同意」，恰好抹掉了争议综述最该保留的信息：持异议的科学家在反对什么、什么样的证据会改变他们的立场。这些不是 Prompt 调优能解决的问题，而是关于「一场科研审查应该怎么组织」的结构性问题。

计算社会科学让这些风险变得具体。复现危机显示大量已发表发现无法复现¹；发表偏差夸大表观效应量²；研究者自由度成倍放大假阳性³。在本项目聚焦的领域——数字行为、社交媒体与心理健康——横截面设计被例行地报告为因果、自报告测量占据主导、少量数据集支撑着大量文献⁴。

1.2 对策：不是更快的综述机器，而是一面照出证据裂缝的透镜

Poliscope 是这个问题的回答。它是一款面向计算社会科学争议问题的深度研究智能体：输入一个存疑的问题，它组织 **7 名各有专长的 AI 科学家**独立取证、互相质询、专门找反例和漏洞，最终产出一张结论与局限并排、证据可溯源、异议不删除的争议证据地图。

Poliscope 的核心方法叫 **EpistemoBrain**。它不是「7 个聊天机器人加 1 个总结机器人」——那是本项目的设计者们反复强调的误区。本项目的立场是：

「让 7 个 Agent 轮流各说一段话、再让第 8 个 Agent 写摘要，做出来的东西看起来热闹，但解决不了任何一个真实的科研盲点问题——分歧会在总结那一步被拍平，没人的判断真正被别人的证据挑战过。」

因此 EpistemoBrain 的架构起点是反过来的：先决定「一个可靠的争议审查该有的组织结构和记忆机制是什么样」，再把 7 个角色放进这套结构里。整个设计的核心论点可以压缩为一句话：

¹Open Science Collaboration, *Estimating the reproducibility of psychological science*, Science, 2015.

²D. Fanelli, “Positive” results increase down the hierarchy of the sciences, PLoS ONE, 2010.

³A. Gelman & E. Loken, *The garden of forking paths*, 2013; B. Nosek et al., *The preregistration revolution*, PNAS, 2018.

⁴A. Orben & A. Przybylski, *The association between adolescent well-being and digital technology use*, Nature Human Behaviour, 2019; P. Valkenburg et al., *Social media use and its impact on adolescent mental health*, Current Opinion in Psychology, 2022.

可靠的盲点发现是一个组织与治理问题，不是一个 Prompt 问题。
异议必须靠构造保留，证据必须靠构造审计，过程必须靠构造不能变成证据。

1.3 三点贡献

1. **七人科学议会 (Council Protocol)**: 7 名专业化科学家通过结构化科研动作（提出、支持、质询、限定、分叉、请求、修正、异议）交互，在明确禁止多数投票的前提下，产出保留少数意见的条件化共识。
2. **双图证据治理 (Dual-Graph Governance)**: 过程记忆图与正式证据图物理分离，由一本追加写、幂等、可重放的科学事件账本连接；唯一的图投影器写入权限在数据库层强制。
3. **可验证的实现与评测**: 开源实现 (MIT)，配合 ForesightBlindspot 时间切片评测协议、五条基线与六项消融，为计算社会科学的盲点发现建立测量基础。

1.4 本白皮书的组织

本白皮书面向想彻底理解 Poliscope 底层机制的技术读者。第 2 章先给出核心方法 EpistemoBrain 的整体轮廓——三根支柱与认识论循环，让读者先看清「我们信什么」；第 3 章给出系统架构总览；第 4 章详述七人议会协议；第 5 章讲三层记忆与 MemoBrain 适配；第 6 章讲双图架构与事件账本；第 7 章讲证据门、三层审计与证据谱系；第 8 章专题讨论 *token* 与成本控制——七人全程参与的系统如何做到不烧钱；第 9 章讲状态机、快照与可靠性；第 10 章讲 ForesightBlindspot 评测；第 11 章讲工程实现细节。

文档中所有事实均来自仓库当前代码（‘packages/’ 与 ‘apps/’），并标注关键文件与符号，便于读者对照源码核验。诚实起见：文档会明确区分「已完整落地」与「规格中自注 MVP 范围外的保留项」。

2 EpistemoBrain: 核心方法体系

2.1 方法定位：一个组织，不是一个聊天机器人群

EpistemoBrain 是 Poliscope 的核心方法。第 1 章指出了三个失效模式（单一视角、一致不是证据、总结器拍平异议），EpistemoBrain 就是对这三个失效模式的系统回答。它的立场先于实现：

可靠的盲点发现是一个组织与治理问题，不是一个 Prompt 问题。

因此 EpistemoBrain 不是「7 个 Agent 轮流发言 + 1 个 Agent 写总结」。那样的结构把分歧留在总结这一步被拍平，没人的判断真正被别人的证据挑战过。EpistemoBrain 从「一场可靠科研审查应有的组织结构」出发，把 7 个角色放进一个强制质询、强制记忆、强制审计的框架里。

2.2 三根支柱

整个方法由三根支柱构成，分别回答「谁在研究」「靠什么记得」「什么算证据」：

1. **七人科学议会（Council Protocol）**——回答「谁在研究」。7 名各有专长的 AI 科学家通过结构化科研动作交互（提出、支持、质询、限定、分叉、请求、修正、异议），在明确禁止多数投票的前提下，产出保留少数意见的条件化共识。这是对「单一视角」和「一致不是证据」两个失效模式的直接对抗：多视角不是靠人多，而是靠每个视角被制度化地互相挑战。机制细节见第 4 章。
2. **三层记忆（Three-Layer Memory）**——回答「靠什么记得」。私人记忆、集体执行记忆、争议证据地图三层闭环：过程产生证据，证据暴露盲点，盲点驱动下一轮调查。集体执行记忆（MemoBrain）不是只存档的笔记本，而是主动调度下一轮调查的执行控制层。机制细节见第 5 章。
3. **双图证据治理（Dual-Graph Governance）**——回答「什么算证据」。过程记忆图与正式证据图物理分离，由一本追加写、幂等、可重放的科学事件账本连接；唯一图投影器写入权限在数据库层强制。过程不自动成为证据是这条支柱的核心约束。机制细节见第 6、7 章。

2.3 一个研究任务的认识论循环

把三根支柱放进一次真实研究，得到一条完整的认识论循环：

研究者确认原子主张

-> 7 席独立预承诺（先行封存判断，避免后续一致是锚定）

-> 专业取证（每席从各自维度发证据请求，合并去重后检索）

-> 证据交换（交换结构化证据，不是完整思维链）

-> 交叉质询（暴露隐藏假设；被质询者只能 DEFEND/REVISE/NARROW/WITHDRAW/DISSENT）

-> 盲点悬赏（证据图暴露的缺口生成盲点候选，按影响 x 不确定性
 x 可调查性 x 新颖度 - 成本排序，7 席共同认领）

-> 联合建模（组合为条件化共识；最强反对或证伪条件缺失时拒绝出共识）

-> 最终复判（7 席再次独立判断，记录信念更新与具名异议）

-> 争议证据地图（结论与局限并排、证据可溯源、异议不删除）

这个循环的每一环都对应一个防止特定失败的结构：预承诺防止锚定，取证防止空谈，证据交换防止重复阅读，交叉质询防止隐藏假设被放过，盲点悬赏把「没人碰过的缺口」显式化，联合建模防止多数票冒充共识，最终复判防止信念在压力下漂移后无人记账。

2.4 认识论立场：三个「不」

EpistemoBrain 的认识论立场可以用三个「不」概括，它们是方法层的硬约束，不是装饰：

1. **不投票：**禁止多数投票直接裁决科研真理。最终结果必须是条件化共识，并保留少数意见、真正分歧和解决分歧所需证据（见第 4 章）。

2. 不匿名删除异议:被反驳、隔离或压缩的观点必须保持可追溯。`DebateCapsule` 与 `DissentCertificate` 让「被反驳过什么、谁持异议、为什么」永远留在正式证据图里（见第 7 章）。
3. 不把过程当证据: 7 名科学家不得直接写证据图；所有正式修改必须先写入事件账本，再由唯一的图投影器投影。思维链、工具记录、失败路线都属于过程材料，永不自动升级为正式证据（见第 6 章）。

2.5 与普通 Deep Research / Multi-Agent Debate 的本质区别

方法层最后值得明确与既有路线划清界限。下表对比 EpistemoBrain 与两种常见实现：

	Deep Research	Multi-Agent Debate	EpistemoBrain
多视角来源	单 Agent 换 Prompt	多 Agent 自由发言	7 席按角色规格、用结构化动作质询
异议去向	被总结器合并	多数胜出	具 名 保 留 (<code>DissentCertificate</code>)
记忆	窗口内上下文	独立、无共享	三层记忆，集体记忆主动调度下一轮
证据标准	引用存在即算	引用存在即算	证据门三层审计 + 因果升级策略
写证据图的权限	无概念	无概念	数据库层唯一投影器
输出	流畅综述	辩论记录	结论与局限并排的可审计证据地图

一句话:Deep Research 优化「读得全」,Multi-Agent Debate 优化「吵得热」,EpistemoBrain 优化「被质疑过的结论还能被追溯」——后者正是盲点发现所需要的。

后续章节将逐一展开三根支柱的机制细节：第 3 章给系统架构总览，第 4 章讲七人议会，第 5 章讲三层记忆与 MemoBrain 适配，第 6 章讲双图与事件账本，第 7 章讲证据门与三层审计，第 8 章专题讨论 token 与成本控制，第 9 章讲可靠性，第 10 章讲 ForesightBlindspot 评测，第 11 章讲工程实现。

3 系统架构总览

3.1 模块化单体

Poliscope 采用模块化单体（modular monolith）+ 异步 Worker，首版刻意不做微服务拆分——这是 YAGNI 约束的直接结果：MVP 只实现能证明「EpistemoBrain 比普通研究智能体更可靠地发现高价值科研盲点」这一主张的功能，微服务拆分不在其中。仓库按职责划分模块：

应用层四个入口共享同一套后端契约：`apps/api`（FastAPI）、`apps/worker`（任务认领与议会执行）、`apps/cli`（纯 HTTP 客户端，不 `import packages`）、`apps/web`（React 证据工作台）。CLI 是纯 HTTP 客户端是一条硬约束——任何第二条直接进入研究服务的代码路径都会绕过证据门。

模块	职责
packages/kernel	冻结契约（ContractModel/FrozenDict）、数据库、配置、权限常量；被所有模块依赖，不依赖任何模块
packages/council	7 个席位、结构化动作、8 个轮次、轮次注册表、席位--网关桥
packages/epistemo	编排器、状态机、预算、停止条件、恢复
packages/memory	MemoBrain 适配器、席位私有记忆、Process Graph、证据投影
packages/evidence	事件账本、证据门、图投影器、谱系、独立性、辩证折叠
packages/papers	取证服务、候选池、查询规划、解析、Paper Packet
packages/models	模型网关（审计装饰器 + 录制回放）
packages/tools	工具网关、四个学术数据源适配器
packages/reports	Research Brief 组装、Markdown / JSON 渲染、导出脱敏
packages/research	研究契约、主张原子化、任务仓储与应用服务
packages/evaluation	ForesightBlindspot 基准语料与验收矩阵

3.2 一次研究的生命周期

一次研究从研究者提交 `Research Contract` 开始：

- 建任务：**研究者提交问题、范围、预算、可选知识库与 PDF。系统原子化出建议的原子主张（`suggested_claims`），任务停在 `AWAITING_CLAIM_CONFIRMATION`——研究者控制方向，系统不自作主张开跑。
- 确认主张：**研究者勾选要调查的主张，任务入队（`QUEUED`）。被放弃的主张标记为 `DISCARDED` 而非删除（审计）。
- Worker 认领：**`SELECT ... FOR UPDATE SKIP LOCKED` 从队列认领，多开安全。
- 议会执行：**7 席 × 8 阶段跑完一次未提交事务。每一轮每一席通过 `GatewayDeliberator` 与模型网关交互，输出结构化科研动作，追加到科学事件账本。
- 图投影：**投影器以第二身份、第二事务消费账本事件，过证据门，写入证据图。
- 报告：**任务终态从图、账本、任务行组装 `Research Brief` 与最终论文。

3.3 为什么拆两个事务、两个身份

「投影器是证据图唯一写入者」如果只写在文档里，就只是一句愿望。Poliscope 用数据库权限把它变成事实：应用身份（`poliscope_app`）在数据库层没有写证据图表的权限；投影器身份（`poliscope_projector`）在数据库层没有写账本的权限。两边都无法伪造对方的数据。这个隔离由迁移与 `packages/kernel/privileges.py` 实现，并有 10 个集成测试用例逐条验证（`test_graph_write_isolation.py`）。

3.4 为什么队列在 PostgreSQL 而不是 Redis

认领任务和更新状态在同一个事务里，Worker 崩溃时锁自动释放。用 Redis 会多出一个可能与数据库不一致的「谁在跑」的真相来源。Poliscope 首版没有任何组件依赖 Redis 做缓存

研究者 → Council (7 seats × 8 phases) → Event Ledger → Evidence Gate →
Graph Projector → Evidence Graph → Blindspot bounties → Council (loop)

图 1: EpistemoBrain 架构：议会、账本、证据门、投影器与认识论路由形成闭环。过程记忆（左，每席 MemoBrain）与正式证据图（右）物理分离。

或广播——加一个没人用的服务违反 YAGNI 约束。这一取舍记录在 `apps/worker/main.py` 的模块文档中。

4 七人科学议会

4.1 七个席位

每个正式研究任务,7 名科学家全程参与。它们在 `packages/council/contracts.py:9-17` 的 `Seat` 枚举中定义:

席位	中文名	核心职责
<code>theory_builder</code>	理论建构者	机制与风险预测、竞争理论
<code>causal_scientist</code>	因果推断专家	混杂、反向因果、选择偏差、识别策略
<code>measurement_scientist</code>	测量与构念专家	构念与操作化的距离、自报告偏差
<code>replication_scientist</code>	统计与复现专家	功效、精度、复现史
<code>boundary_scientist</code>	边界与情境专家	适用与不适用的范围、异质性
<code>adversarial_falsifier</code>	对抗性证伪者	攻击最强版本、找反例
<code>evidence_auditor</code>	证据与溯源审计员	锚点、独立性、引用蕴含

EpistemoBrain 是无投票权的组织脑——不是第 8 名科学家，也不代表任何学术立场。

7 个席位共享运行框架和工具缓存,但必须拥有独立的东西:独立角色规格(`packages/council/roles.py` 的 `ROLE_SPECS` 定义每席 `expertise`)、独立私有记忆 (agent id 为 `{task_id}:{seat}`, 硬约束无人读他人 id)、不同证据投影权重 (`packages/memory/projection.py`)。这三样都有测试断言它们确实不同——曾有五个席位共用一个空权重表,在唯一用于区分它们的维度上其实是同一个 Agent 的五份拷贝。

单一运行框架: `packages/council/deliberation.py` 用一个 `GatewayDeliberator` 类 + 7 套席位指令服务 7 席,满足「不复制 7 套业务代码、不部署 7 个服务」的约束。

4.2 八个阶段 (PHASE_SEQUENCE)

任务按 `packages/epistemo/contracts.py:75-84` 的 `PHASE_SEQUENCE` 顺序执行:

PRECOMMITMENT -> ACQUISITION -> EVIDENCE_EXCHANGE -> CROSS_EXAMINATION
-> BLINDSPOT_BOUNTY -> JOINT_MODELING -> FINAL_REJUDGMENT -> REPORTING

1. **独立预承诺 (Independent Precommitment):** 7 席分别提交初始判断、置信度、三个最可能的盲点、需要补充的证据、什么证据会改变判断。封存之后才能读取，读取之后不能提交——这是「后续一致不是锚定」的唯一保证。
2. **专业取证 (Professional Acquisition):** 7 席从各自维度发出证据请求，由查询规划器合并去重，执行批量检索。证据需求矩阵 (Evidence Demand Matrix) 驱动搜索行为，而不是泛泛的关键词。
3. **证据交换 (Evidence Exchange):** 交换的是结构化证据 (主张、证据原文、来源、研究设计、证据质量、适用边界、置信度)，不是完整思维链。
4. **交叉质询 (Cross-Examination):** 目标不是争胜而是暴露隐藏假设。被质询者只能选择 DEFEND/REVISE/NARROW/WITHDRAW/DISSENT。
5. **盲点悬赏 (Blindspot Bounty):** 证据图暴露的缺口生成盲点候选，按影响 $\times$ 不确定性 $\times$ 可调查性 $\times$ 新颖度 - 成本评分，广播给 7 席认领。
6. **联合建模 (Joint Modeling):** 把各席调查结果组合成条件化共识。禁止多数投票裁决科研真理——在「最强反对」或「证伪条件」缺失时拒绝出具共识 (registry.py:1823-1852)。
7. **最终复判 (Final Rejudgment):** 7 席再次独立判断，记录信念更新与具名异议 (DissentCertificate)。
8. **报告生成 (Reporting):** 从图、账本组装 Research Brief 与最终论文。

一个席位失败不终止任务。失败被记为该轮的 PHASE_FAILED 事件和一个未填槽位，下一轮照常执行，任务以 COMPLETED_WITH_GAPS 收尾。

4.3 结构化科研动作

科学家只允许提交结构化动作 (packages/council/contracts.py:22-38):

PROPOSE	SUPPORT	CHALLENGE	QUALIFY	FORK	REQUEST	REVISE	DISSENT
DEFEND	REVISE	NARROW	WITHDRAW	DISSENT	(收到质询后)		

每个动作必须包含 actor、target、statement、evidence_refs、confidence、falsification_condition、novelty。以下内容被账本拒绝：没有目标节点的泛泛评论、没有来源的事实性主张、与已有内容高度重复的观点、只有「我同意」但没有新增信息的发言、无法说明如何被推翻的绝对断言。动作校验在 packages/council/actions.py。

4.4 禁止多数投票：条件化共识

evaluate_consensus 中,即使多数席位 SUPPORT,只要 has_unresolved_fatal_challenge 为真或 evidence_refs 为空，共识即被 BLOCKED。联合建模阶段的 JointModelingHandler 在「最强反对」或「证伪条件」缺失时拒绝产出共识——注释直接声明这是多数投票禁令的机制。

最终输出不是「6:1 通过」，而是：

多数判断：存在小幅、条件性的影响。

少数异议：现有研究仍不足以排除反向因果。

真正分歧：是否将纵向时间顺序视为足够的因果证据。

解决分歧所需证据：具有平台日志和基线心理健康控制的预注册纵向研究。

这就是条件化共识，而不是多数统治。

4.5 人类方向性引导检查点（AWAITING_COUNCIL_INPUT）

盲点悬赏（第 5 轮）结束、联合建模（第 6 轮）开始之前，存在一个可选的人类方向性引导检查点：任务状态进入 `AWAITING_COUNCIL_INPUT`，研究者可以查看 7 席在盲点悬赏结束时的立场（预承诺、置信度、已提出质询），并提交一段方向性备注，或明确留空直接继续。

这个检查点不构成对「禁止多数投票」的违反：它是单一研究者的方向性输入，不是投票计数；只影响联合建模阶段「接下来重点讨论哪些悬而未决的冲突」这类调度性内容；不改变任何 Evidence Gate 判定逻辑，不作为任何 Claim 的 SUPPORTS/REFUTES 证据来源，不进入 DebateCapsule 或 DissentCertificate 的构造字段。联合建模的 prompt 会把它作为一段独立、明确标注来源的文本注入（例如“[研究者方向性备注，非科学判断]: ...”），供模型参考，不得被模型当成第 8 名科学家的科研判断。

宽限期（默认 120 秒，`POLISCOPE_COUNCIL_INPUT_GRACE_SECONDS` 配置，最小 30 秒）届满仍未操作时，服务端自动把任务置回排队，以「无方向性干预」继续——研究不会被研究者走开而卡死。

实现：`TaskStatus.AWAITING_COUNCIL_INPUT(epistemo/contracts.py:28)`、`CouncilCheckpoint.guid` HTTP 端点 `GET/POST /api/tasks/{id}/council-preview` 与 `/council-guidance`、CLI 子命令、前端 `CheckpointGate.tsx`。

5 三层记忆与 MemoBrain 适配

5.1 记忆不是私有缓存，而是公共认知层

Poliscope 的架构源自一个判断：记忆不是每个智能体的私有缓存，而是科研共同体的公共认知层。它由三层组成，各回答一个问题：

1. **私人记忆（Private Memory）**——每位科学家自己的探索轨迹、工具记录、未验证的猜想，只服务自己、不被他人直接读取。回答「这位科学家是怎么走到这一步的」。
2. **集体执行记忆（Collective MemoBrain）**——整个共同体发生的结构化认知事件：谁提出过什么主张、谁攻击过谁、哪些争议还悬着、哪些盲点没有人碰过。回答「我们研究到哪一步了、下一步该查什么」。它不是只存档的笔记本，而是主动调度下一轮调查的执行控制层。
3. **争议证据地图（Evidence Map）**——面对用户的正式知识：只存放经过审计的原子主张、研究发现、构念、情境和盲点。回答「我们现在对这个问题知道什么」。

三层之间不是堆叠，而是闭环：过程产生证据，证据暴露盲点，盲点驱动下一轮调查。

5.2 MemoBrain 集成规则

MemoBrain 是外部方法基座，代码仓库为 github.com/qhj00/MemoBrain。集成前核验了其许可证，并通过 `MemoBrainAdapter` 适配，避免修改上游源代码。首版适配接口保持小而清楚，仅五个方法：

```
init_private_memory(agent_id, task)
memorize_episode(agent_id, episode)
recall_private(agent_id, token_budget)
save_snapshot(agent_id)
load_snapshot(agent_id)
```

接口定义在 `packages/memory/contracts.py`。诚实说明：当前 `packages/memory/adapter.py` 的 `create_memory_adapter()` 返回的是 `InMemoryMemoryAdapter`（内存字典替身），尚未真实导入上游 MemoBrain 代码；Process Graph 的 Flush/Fold/Recall 为简化占位实现。这是规格自注的保留项，不是已完成的集成。

5.3 席位私有记忆：一个规则保证隔离

`packages/memory/council_memory.py` 用一个规则满足「独立私有记忆」：agent id 是 `{task_id}:{seat}`，没有任何代码读另一个席位的 id。同一任务的两个席位互相看不到对方的 recall；同一席位在两个任务之间也不携带一个任务的记忆进入另一个。

Recall 预算：DEFAULT_RECALL_BUDGET = 800 字符。这刻意设得很小——recall 与证据投影竞争同一份上下文，它的设计目标是保留科研骨架而不是重放完整转录：当前任务、已确认 Finding、活跃 Blindspot、未解决质询、少数异议、下一步证据需求。800 字符能保住这些，而不会把一份不断增长的转录灌进后面每个席位的 prompt。

`CouncilMemory` 负责：`open`（给每席播种任务）、`remember`（记住一个 episode）、`recall`（读回各自 recall）、`snapshot/restore`（暂停任务的记忆恢复）。

5.4 过程记忆的 Fold：保骨架不保字数

`packages/memory/fold.py` 的 `fold_process` 只接受 Process Graph 快照，任何尝试折叠证据图节点的行为抛 `GraphBoundaryViolation`。它按 Task/ToolCall/Decision 三类节点压缩，但压缩前必须通过 `BackboneRetention` 的六项骨架保留校验：

```
当前任务、已确认 Finding、活跃 Blindspot、未解决质询、
少数异议、下一步证据需求
```

只有六项全部保留，折叠才算成功（`FoldResult.rejected` 否则为 True）。这是过程记忆折叠的硬性约束：压缩任何过程节点都不得丢失科研骨架。

5.5 Perspective Recall：角色化召回

`packages/memory/recall.py` 的 `perspective_recall` 让不同席位看到同一证据快照的不同排序：按该席位的 `evidence_sort_weights` 排序。因果专家看到的是混杂变量和反

向因果攻击优先，测量专家看到的是构念冲突和测量不变性问题优先。形式上：

$$\text{Context}_i = \text{ProcessRecall}_i + \text{EvidenceProjection}_i$$

其中 ProcessRecall_i 来自该席自己的私有记忆（已完成什么、失败过什么、当前任务、下一步计划）， $\text{EvidenceProjection}_i$ 是从证据图生成的角色化视图。这避免多智能体两个常见问题：每个 Agent 重复阅读所有内容；所有 Agent 最后产生相同观点。

5.6 认识论路由：让记忆图主动调度科学家

MemoBrain 原本主要负责「记什么、忘什么」。Poliscope 进一步让它决定「下一轮应该派哪个科学家调查什么」。证据图发现缺口后生成盲点悬赏，按影响、不确定性、可调查性、新颖度、成本打分排序，广播给 7 席认领——是证据状态驱动下一步调查方向，而不是主持人按预定台本点名。

证据图发现缺口 -> 生成盲点悬赏 -> 选择最匹配的科学家
 -> 分配角色化上下文 -> 执行检索/验证 -> 结果写回集体记忆
 -> 重新计算认知前沿

这不是「挑出一个最优秀的科学家由它独自解决」，而是 7 席全程在场、从不同视角共同调查同一个盲点（分工表在 `registry.py` 的盲点悬赏轮中生成）。

6 双图架构与事件账本

6.1 为什么是两张图

MemoBrain 的推理图回答「科学家是怎样完成这次研究的」，争议证据图回答「我们对这个研究问题目前知道什么」。两者不能混为一张图，否则：工具调用步骤与学术证据混在一起、Agent 的中间猜想被误认为事实、Fold 推理过程时误删证据、一篇论文中的多个研究发现无法精确表示、界面无法清晰展示争议结构。

因此 Poliscope 采用**双图模型**：

1. **Process Graph**（过程图）：记录科学家的任务、工具调用、失败路线、质询、决策。由 MemoBrain 管理，允许普通 Flush/Fold/Recall。过程节点不会自动成为正式科研证据。
2. **Evidence Graph**（证据图）：10 种节点、12 种边，只存放经过审计的正式知识。只有 Graph Projector 能写。

6.2 证据图：首版正式节点

ResearchQuestion Claim Source StudyFinding Construct

Context Blindspot DebateCapsule DiscriminatingStudy DissentCertificate

在 `packages/evidence/contracts.py:10-35` 定义。`DebateCapsule` 由联合建模阶段的 `Dialectical Fold` 产出, `DissentCertificate` 由最终复判阶段为持异议的席位产出——两者都是「异议不得被静默删除」原则的直接实现: 被反驳、隔离或折叠的观点必须保持可追溯。

6.3 证据图: 首版正式边

SUPPORTS REFUTES QUALIFIES CONTRADICTS CONFOUND MEDiates
MODERATES OPERATIONALIZES DERIVED_FROM APPLIES_IN EXPOSES TESTS

每条边也是可审计对象: `source`、`target`、`strength`、`rationale`、`scope`、`created_by`、`reviewed_by`、`status`。

6.4 科研事件账本 (Scientific Event Ledger)

7 名科学家不得直接写证据图。所有正式修改必须先写入 `packages/evidence/sql_ledger.py` 的 `SqlEventLedger`。关键机制:

1. 幂等可重放: `uq_event_idempotency` 唯一约束 + 全序 `sequence(coalesce(max(sequence),0)+1)`。重跑一次是空操作。
2. 审计防线: 同一幂等键、不同 payload 抛 `EventConflict`——防止任何改写历史的尝试被静默接受。
3. 串行化: `pg_advisory_xact_lock` 保证并发无乱序。
4. 过程事件不升级: `on_conflict="skip"` 仅用于 `process-only` 事件。

幂等键由阶段 + 席位 + 位置派生 (`PhaseContext.key()`, `registry.py:451-480`, 含超长 query 的 hash 折叠——这是修复过真实生产事故的细节)。任何把 `uuid4()` 写进 payload 的轮次都会在重放时与自己的幂等键冲突, 有测试专门盯这件事。

6.5 Graph Projector: 唯一写入者

`packages/evidence/sql_projector.py` 的 `SqlGraphProjector` 是证据图的唯一写入者。关键机制:

1. `NODE_EVENT_TYPES allowlist`: 拒绝过程事件升级为证据。
2. `node_id_for`: 节点跨重放保持 id 稳定。
3. `__write_edges`: 对悬空目标抛 `ProjectionError`——不允许指向不存在节点的边。
4. `checkpoint` 单调不回退。

双图隔离在数据库层强制: `privileges.py` 中 `app` 角色对 `graph` 表仅 `SELECT`, 投影器角色可 `INSERT/UPDATE` 但无 `DELETE` (迁移 0019 仅对会话删除放行), 账本表 `app` 角色仅 `APPEND`。集成测试 `test_graph_write_isolation.py` (10 用例) 逐条验证。

6.6 双图如何双向联动

方向 1：过程图 → 证据图。每次科学家完成一个 episode：思考 → 调用检索工具 → 阅读论文 → 提取研究发现 → 提交 SUPPORT 事件。Private MemoBrain 把整个过程记入 Process Graph，但只把结构化事件送入证据图验证管道。过程图保存「为什么搜索、搜了什么、失败了什么」，证据图保存「论文、具体发现、支持关系、适用边界、审核状态」。

方向 2：证据图 → 过程图。证据图发现新盲点（例如「支持 C2 的 7 项研究发现全部使用自报告数据」），生成 Blindspot，然后创建新的过程任务 ASSIGNMENT，写入 7 席的 Private MemoBrain，开始新一轮推理。形成闭环：

过程产生证据 -> 证据暴露盲点 -> 盲点产生新任务 -> 新任务产生新过程 -> 新过程更新证据

6.7 被反驳、隔离、折叠的节点不物理删除

SqlGraphProjector 的 upsert 不删节点，只改 `status.DebateCapsule.__post_init__` (`evidence/dialectical_fold.py:22-34`) 要求正反双方字段全非空，否则拒绝折叠。`DissentCertificate` 挂在事件本体上，防止报告摘要时丢失。前端 `MapView` 注释直述「dimmed, never hidden」——被反驳的节点淡化但永不消失。对应测试 `test_dissent_preservation.py`、`test_dialectical_fold.py`。

7 证据门、三层审计与证据谱系

7.1 六阶段证据门

每个候选事件在 `packages/evidence/gate.py` 的 `FullEvidenceGate` 中依次通过六个阶段：

SCHEMA -> DEDUPLICATION -> SOURCE -> CITATION_ENTAILMENT
-> METHOD_QUALITY -> GRAPH_CONSISTENCY

1. **Schema 校验：**字段是否完整、类型是否正确。
2. **语义去重：**与已有对象是否重复。
3. **来源验证：**DOI 是否解析、标题/作者/年份是否一致、是否撤稿。
4. **引用蕴含：**原文是否真的支持主张。
5. **方法质量：**直接性、设计强度、测量效度、统计精度、可复现性、外部效度。
6. **图一致性：**是否破坏图约束（矛盾节点、重复分叉谱系）。

关键实现细节：`GRAPH_CONSISTENCY` 阶段真正调用 `consistency.py::check_graph_consistency()` (`gate.py:378-395`)，判定所需布尔值来自 `SqlGraphConsistencyQuery` 对数据库会话的真实查询——不是恒真的空检查。第 3 和第 5 阶段读取候选事件自己带的元数据，撤稿会被拒。审计发现逐条写入 `EventAuditModel`。

7.2 证据分层 A-D

等级	含义	处置
A	全文与精确原文可得	ADMIT → 节点 active
B	只有摘要和可靠元数据	SOURCE_ONLY → 节点 provisional
C	综述中的二手描述	DISCOVERY_ONLY → 留在账本，不入图
D	网页或新闻线索	TOOL_LEAD_ONLY → 留在账本，不入图

Level B 不得单独支撑高置信因果结论；Level C 和 D 不得替代原始研究。目前的取证管线只能拿到元数据，所以只产 Level B——这不是配置问题，是实事求是：把元数据标成 Level A，就等于允许一个高置信因果结论建立在摘要之上。

7.3 三层审计

正式 Finding 必须经过三层审计(`source_verifier.py`、`citation_verifier.py`、`method_auditor.py`):

1. 来源真实性：DOI、标题、作者、版本、撤稿状态、PDF 身份。
2. 引用蕴含：引言假设不得报告为实测结果；讨论段的推测不得写成实证发现；「不显著」不得写成「没有效应」；相关不得升级为因果；子群结果不得外推为总体结论。
3. 方法质量：直接性、设计强度、测量效度、统计精度、可复现性、外部效度。

7.4 因果越级拦截

`packages/evidence/causal_policy.CausalUpgradePolicy` 在证据门第六阶段专门拦截「横截面设计 + 因果主张」的组合，事件被隔离并留下审计记录——相关不自动升级为因果。这是把「相关不等于因果」从一句提醒变成机制的落实。

7.5 证据谱系与独立性

`packages/evidence/lineage.py` 的 `LINEAGE_DEPENDENCY_TYPES` 定义 8 种依赖类型：

`SAME_DATASET` `OVERLAPPING_SAMPLE` `SAME_RESEARCH_TEAM` `PREPRINT_VERSION_OF`
`EXTENSION_OF` `REANALYSIS_OF` `CITES_WITHOUT_NEW_DATA` `META_ANALYSIS_INCLUDES`

`packages/evidence/independence.py` 的 `cluster_evidence` 用并查集聚合独立证据簇。共享研究团队不合并——同一个实验室的两个数据集仍然是两份证据。报告同时显示论文数与独立证据簇数：六篇论文如果共享同一份数据集，只算一份独立证据。

7.6 防止共享证据错误的四种机制

1. 盲证据评审 (Blind Evidence Review): 第一轮判断中剥离作者、期刊、被引量等声誉信号,只给方法、样本和结果。当前仅有结构性测试(`test_deliberation_blind_review.py`) 锁定 `GatewayDeliberator` 不透传声誉字段; 规格自注 MVP 范围外, 无 round handler 主动剥离。

2. **独立双抽取 (Independent Extraction)**: 对高影响论文由两个不同模型/角色独立抽取并比较。当前无实现, 规格自注 MVP 范围外。
3. **来源多样性约束 (Source Diversity Constraint)**: `packages/evidence/source_diversity.py`
当一项 Claim 的所有支持证据来自同一数据集/团队/国家/测量方式/设计时, 自动生成 Blindspot, 不给高置信结论。
4. **对抗式检索 (Adversarial Retrieval)**: `packages/evidence/adversarial_retrieval.py`
为每个 confirmed claim 生成 6 类反向检索意图 (反驳、零结果、替代理论、测量批评、复现失败、边界翻转), 经工具网关发起真实查询。查不到的诚实记录在 `ADVERSARIAL_RETRIEVAL_ATTEMPTED` 事件的未解析计数里——不伪造命中。

8 token 与成本控制：七人全程如何不烧钱

七人全程参与确实比动态激活昂贵。Poliscope 用六层工程机制控制成本, 而不是靠减人——「任何一次省略某个角色的调度, 都在赌这个角色这次用不上」是被否决的方案。以下逐层拆解。

8.1 发言预算：每轮每席有上限

每一轮、每一席最多一个主动作 + 一个质询 + 一个回应; 没有新增信息就 PASS。这直接限制了一轮议会内模型调用次数的上界。实现上由各轮 handler 的收集逻辑 (`registry.py` 的 `_collect`) 约束, 一个席位一次 phase 至多触发有限次模型调用, 超出即记缺席。

8.2 新颖性门控：先去重再推理

提交动作前先与现有证据图进行语义去重: 与已有节点相似度超过阈值且没有新增证据/边界/攻击, 就不调用高成本推理、不写入集体图。`packages/evidence/gate.py` 的 `DEDUPLICATION` 阶段就是这道闸门。一个来源去重命中已持久化行时, 不会重复花费工具/模型预算 (`registry.py`:927-929 注释)。

8.3 分层模型调用：复杂判断用强模型，机械操作用轻量模型

`packages/council/deliberation.py`:125-132 的 `PHASE_MODEL_CLASSES` 按阶段分配模型档位:

```
PRECOMMITMENT -> STRONG_REASONING # 独立判断, 质量关键
ACQUISITION    -> MEDIUM           # 检索意图生成
EVIDENCE_EXCHANGE -> MEDIUM
CROSS_EXAMINATION -> STRONG_REASONING # 质询, 质量关键
BLINDSPOT_BOUNTY -> STRONG_REASONING # 盲点评分, 质量关键
JOINT_MODELING  -> MEDIUM
FINAL_REJUDGMENT -> STRONG_REASONING # 最终复判, 质量关键
```

强推理阶段（预承诺、交叉质询、盲点悬赏、最终复判）用强模型，机械阶段（取证、交换、联合建模）用轻量模型。部署时通过 `POLISCOPE_MODEL_NAME_STRONG_REASONING / _MEDIUM / _LIGHTWEIGHT` 分别配置，模型名留空时按序回退到任务自带配置 → 部署默认 → `deepseek-v4-flash`。

8.4 共享检索缓存：七人合查一个池

七名科学家不各搜一遍相同关键词。他们提出证据需求，`Query Planner` 合并去重检索意图，`Retriever` 执行批量检索，所有结果进入统一候选池（`Shared Discovery Pool`）。同一论文：PDF 只下载一次、文本只解析一次、DOI 只验证一次、元数据统一缓存；不同角色读取同一篇文章的不同字段和片段。共享缓存的同时，每个角色仍基于自己的视角筛选和阅读——去重的是检索成本，不是认知多样性。

8.5 共享缓存与模型调用的可观测成本

所有模型调用走 `packages/models/gateway.py` 的 `AuditedModelGateway`（装饰器），`models/audit.py` 的 `record_model_call` 把每次调用落库：

```
input_tokens  output_tokens  cost_usd  latency_ms  retries  schema_status
```

工具调用同理走 `AuditedToolGateway`。费用/延迟/重试/错误全部可审计——模型与工具调用的每一笔成本都必须可解释，也让研究者能看见钱花在了哪里。

8.6 四维预算：不是只卡一个数

`packages/epistemo/budget.py` 的 `BudgetTracker` 同时追踪四个维度：

```
wall_clock_minutes (总时长)  model_cost_usd (模型费用)
tool_call_limit (工具调用数)  source_limit (来源数)
```

每个消费者在花钱之前检查预算（`consume_*` 在扣减前判断剩余）。预算耗尽抛 `BudgetExhausted`，运行以 `COMPLETED_WITH_GAPS` 收尾，未完成证据槽位被列出——预算不足被显式报告，不伪造完整结果。

8.7 阶段级 deadline：防止单个席位拖死全体

`registry.py:493-520: _COLLECT_DEADLINE_SECONDS`（默认 15 分钟）限制整个收集轮；`MAX_SEAT_ATTEMPTS = 2` 限制单席重试次数。一个席位调用超时被记缺席（`SEAT_UNAVAILABLE` 事件带原因与重试次数），下一轮照常执行——不会因为一个席位的模型调用挂起而让整个议会陪跑。模型网关自身还有 240s 流式 + 300s 非流式回退的超时。

8.8 800 字符的 Recall 预算：上下文竞争管理

每席 *recall* 预算 = 800 字符（`council_memory.py::DEFAULT_RECALL_BUDGET`）。recall 与证据投影竞争同一份上下文，刻意设小是为了保留科研骨架（当前任务 + 最近几个阶段摘

要) 而不是重放完整转录。随着阶段推进, 后面席位的 prompt 不会因为前面所有轮次的完整历史而无限膨胀。

8.9 Free Trial 与 BYOK: 成本在研究者手里

模型费用可以由部署方配置(POLISCOPE_MODEL_),也可以由研究者自带(BYOK,任务级 model_config)。免费试用额度 FREE_TRIAL_LIMIT = 2(packages/models/free_trial.py) 在 confirm-claims 时原子扣减。API key 只存服务器/任务行, 任何读端点都不回显, 绝不进仓库、日志或前端。

9 可靠性、状态机与快照

9.1 显式状态机

packages/epistemo/state_machine.py 的 TaskStateMachine 管理任务状态。_transition_to_phase 只允许 current_index+1 的跳转, 跳转抛 InvalidTransition。任务状态包括:

```
AWAITING_CLAIM_CONFIRMATION -> QUEUED -> RUNNING
-> AWAITING_COUNCIL_INPUT -> QUEUED (检查点后) -> RUNNING
-> COMPLETED / COMPLETED_WITH_GAPS / FAILED / CANCELLED
```

9.2 快照 / 暂停 / 恢复 = 「暂停认领」

deliberate() (apps/worker/jobs.py) 把一个任务的完整 8 阶段议会跑在一次未提交事务里——这是「要么完整、要么不跑」的原子性设计, 保证账本与证据图在任何时刻一致(没有可截断的中间持久状态可供半途快照)。

因此 'pause'/'resume' 的语义是队列层面的: 把任务从 QUEUED 挪到 PAUSED, 恢复之前永远不会被 worker 认领(端到端测试 test_a_paused_task_is_never_claimed_until_resumed)。议会层面的固定暂停点是 JOINT_MODELING 前的检查点(第 4 章)。一个任务两种暂停: 还没轮到你的, 用 pause/resume 挡在队列外; 轮到你了想给议会方向, 用检查点。

9.3 检查点与断点续跑

CouncilCheckpoint (JSONB 列) 保存运行状态: run_phases、carried、unfilled、failures、phase_snapshots、restart_from、guidance。first_unfinished_phase() 决定从断点续跑的位置——它把「从未跑到」的阶段也算作未完成, 使 CANCELLED 停在哪就从下一阶段续跑。

终态任务现在也保留 checkpoint (含 8 阶段 phase_snapshots), 使「从断点处研究」能看到哪些协议阶段实际跑过。重新研究 ** 两种模式:

- 从头研究 (mode=full / rerun_fresh): 克隆新任务 (全新 id、全新账本), 从独立预承诺真正重新开始; 仅继承已确认的主张; 原任务保留终态作为审计历史。

- **从断点研究** (mode=first_gap): 从第一个未完成 (失败、跳过或尚未跑到) 的阶段原位续跑。

从头模式必须克隆新任务而非原位重跑: 同一任务从头重跑会与已提交账本事件的幂等键冲突 (EventConflict)。上传的 PDF 对象通过 rehome_uploaded_objects 迁移到新任务, 避免级联删除。

9.4 单席失败降级

一个席位调用失败被 UnavailableDeliverator 转为 SEAT_UNAVAILABLE 事件, 带原因与重试次数。MAX_SEAT_ATTEMPTS = 2 限制重试, 整轮 _COLLECT_DEADLINE_SECONDS (15 分钟) 限制收集。单个科学家失败不终止任务, 任务以 COMPLETED_WITH_GAPS 收尾, 缺席席位在报告的局限一节被逐条点名。对应测试 test_single_seat_degradation.py、test_seat_retry.py。

9.5 停止即时生效

研究者点「停止」时, 如果任务在跑 (RUNNING), 请求写入侧信道表; worker 在阶段间轮询该表, 发现后抛 CouncilCancelled, 任务转 CANCELLED 并保留 checkpoint (含 phase_snapshots) 作为回退素材。_await_or_cancel 用 1s 轮询让停止即时生效 (round-13 修复)。

9.6 重试与超时策略

模型调用: 流式 240s -> 非流式 300s 回退; schema 修复至多 1 次
席位收集: 整轮 deadline 15 分钟; 单席至多重试 2 次
结构化输出: 修复失败后隔离, 绝不写正式图

9.7 会话删除: 研究者可以销毁自己的会话

DELETE /api/tasks/{id} 物理删除任务及全部子记录 (账本、过程流、证据图、审计行)。迁移 0019 为此授予应用角色 DELETE——这是唯一的权限模型例外 (见 DEVELOPMENT.md「设计上的五个硬约束」第 2 条)。会话历史面板支持行内两段式删除与「清空全部」。

9.8 测试: 三层测试纪律

- **单元测试** (tests/unit): 无外部依赖, 约 1 秒跑完。
- **集成测试** (tests/integration): 需要 Docker, 跨真实数据库与真实角色。每个文件都必须真的连数据库 (目录级 confest 强制)。
- **Golden 测试** (tests/golden): 因果蕴含黄金用例。
- **E2E 门禁** (tests/e2e): 发布门禁。

没有 Docker 时集成测试会跳过并说明原因，不会静默通过——一份在没有数据库的机器上依然全绿的测试报告，等于在撒谎。测试纪律：业务规则/状态机/图约束先写测试；修 Bug 必须加回归测试；关键修复用变异测试自证（把修复回退、确认测试失败、再恢复）。

10 ForesightBlindspot 评测体系

10.1 为什么需要专门的时间切片基准

盲点发现的价值在于提前看见未来才被证实的缺口。一个普通的检索质量基准无法度量「系统能否预判哪个方向未来会有高价值发现」。因此 Poliscope 设计了 ForesightBlindspot：一个时间切片评测协议。

每个 case 是一个存在争议的问题 + 一个截止日：系统只能读截止日前封闭语料，gold blindspots 是「后来被高质量研究证实、截止日前有合理的先兆证据、且不依赖完全不可预见的平台/事件/方法」的缺口。泄漏控制内建于协议：封闭截止前语料、禁止未来引用、可匿名化变量、明确披露基础模型参数可能包含未来知识这一无法消除的风险。

10.2 评测目标与五条基线

核心对比保留五条基线，它们是真实 *council* 的配置而非另写的实现，因此每一条相对上一条恰好增加一个能力：

- 1. **Single-Agent Deep Research**: 一个研究者（理论建构者），无协议、无记忆、无门。
- 2. **Fixed Multi-Agent Debate**: 7 席 + 无差别的 deliberator，无每席记忆、无门。
- 3. **Council + Linear Context**: 7 席共享一份无差别的转录。
- 4. **Council + MemoBrain without Evidence Engine**: 7 席有每席记忆但移除证据门。
- 5. **Full Poliscope**: 完整系统。

优先消融（每项删除一个机制）：无独立预承诺、无对抗性证伪者、无证据审计员、普通 Fold、无证据谱系、无 MemoBrain。

10.3 核心指标族

盲点:	recall, precision, high-impact recall, specificity, actionability, foresight distance
证据:	citation existence, citation entailment, evidence independence, causal overclaim rate, full-text coverage
记忆:	compression ratio, scientific backbone retention, contradiction retention, long-horizon drift, recovery quality
协作:	role diversity, debate utility, belief-update quality, dissent preservation, false consensus rate
成本:	cost per valid blindspot, tokens per admitted finding

10.4 评测管道与生产系统同构

`packages/evaluation/harness.py` 从同一套 `CouncilOrchestrator + GatewayDeliverator + CouncilMemory` 接线出五条基线和六项消融，而不是另写一套评测实现——这使消融结论能真实反映生产行为。打分函数复用生产审计规则（因果越级策略、引用蕴含验证器、谱系检测与证据聚类），而不是重新推导平行近似。对应测试 `test_evaluation_harness.py`、`test_evaluation_ablations.py`。

10.5 基线构造正确性：评测管道发现的修复

评测管道第一次运行就暴露了两个构造缺陷并触发修复：

1. **单 Agent 选错席位：**基线原来按有序枚举取字母序第一个席位，恰好是证伪者——而证伪者在脚本化素材里盲点答案最强，导致「单研究者」得分与完整议会一样高（F1 0.8）。修复：单 Agent 基线显式用理论建构者。
2. **复判全席遍历：**最终复判 handler 无条件遍历全部 7 席，为从未参与的席位铸造占位判断。修复：复判尊重参与席位集合。

回归防护：断言单席盲点覆盖严格低于多席覆盖——未来若基线阶梯坍塌立即失败。论文如实报告了这一修正过程，因为「测量仪器的构造是测量的一部分」。

10.6 当前评测的诚实边界

没有编造任何数字。论文与文档如实标注为未来工作：

- 时间切片语料（截止日前封闭语料）尚未采集 → 无独立 `B_test`。
- 人工标注（金标准盲点的 Kappa/Alpha 一致性）尚未产生标注数据——但协议骨架（双人标注 + 仲裁者 + 统计）已实现于 `packages/evaluation/agreement.py`。

真实模型受控对照实验已经运行（`scripts/live_ablation.py`, DeepSeek-V4-Flash 于 OpenAI 兼容端点，11 变体 × 语义裁判），结果与边界见下节「盲点质量裁判」与论文 4.6 节。

当前可复现的脚本化数字来自 `scripts/arbor_eval.py` 在 demo case（无模型调用、无网络、无数据库）上的确定性测量，属于 `B_dev`。脚本化 gateway 的固有边界被如实报告：协议/记忆/证据门对盲点整合的贡献在盲点指标上不可观测（两个消融在脚本化素材上是零效应），它们体现在辅助分数（异议保留、引用蕴含、证据独立性）并要求真实模型运行才能观测——真实模型运行后的结论见下。

10.7 盲点质量裁判：关键词代理到语义评判

真实模型跑消融时暴露了一个测量缺陷：`score_blindspots` 用关键词子串匹配 `gold blindspot`（例如把 `reverse_causation` 拆词后要求逐词出现），而真实模型的自由文本「效应的方向可能反向」根本不含该 token，导致多数变体 recall 塌陷为 0.0——这不是系统能力

为 0，而是测量仪器失效。修复是 `packages/evaluation/semantic_blindspot.py`：用同一个模型网关（不引第二个厂商 SDK）做语义评判，对每个 gold 概念询问「哪几条候选盲点陈述在语义上覆盖了它」，再按与关键词版完全相同的定义导出 recall/precision。

语义评判的诚实边界同样被保留：它仍不是人工金标准（那是 `agreement.py` 未来工作的位置），且判官不可达或结构化输出被隔离时返回 `None`（「未测量」），绝不静默写成 0.0。`scripts/live_ablation.py` 通过 `-semantic` 开关在同一份消融表里并列输出关键词列与语义列（`rec_sem/prec_sem/F1_sem`），使两种裁判的差异本身可审计。对应测试 `test_evaluation_semantic_blindspot.py`。

10.8 真实模型运行的结果与边界

真实模型（DeepSeek-V4-Flash，官方 API 端点）先跑 11 变体单次扫描，端点随即限流两天，12 个 run 被污染（unfilled 跳到 46、语义裁判全部不可达）——这些 run 被弃用而非报告。随后对 5 个关键变体（完整系统、单 agent 基线、三个机制效应最大的消融）各跑 3 次取均值，降方差后的三个发现：

1. **语义裁判修复了测量**：关键词版在自由文本下对完整系统报 recall 0.0；语义版产生可读、非退化分布，precision 呈现真实梯度（0.406–1.000），裁判本身有效。
2. **降方差后不是噪声，是稳定的精度—召回权衡**：完整 Poliscope 的语义 precision 满分 1.000，但 recall 最低（0.286，三次全部恰好覆盖 2/7 个 gold 概念）；所有简化变体（含单 agent 基线）recall 高达 0.809–1.000，precision 反而降到 0.406–0.586。去预承诺三次全部复现 recall 1.000——此前单次运行的观察不是采样假象，而是真实、可重复的效应。unfilled 解释了差距：完整系统平均 12.3 个空槽（模型产不出结构化提名），简化变体只有 1.0–1.7。**协议负担是 flash 级模型的硬约束**：议会的治理机制用 recall 换取 precision，弱模型下 recall 代价主导了 F1。
3. **数据层确定性分数全 run 一致**：citation entailment 0.5、evidence independence 0.667、dissent preservation 1.0 在 15 个 run 中全部相同——评分管道与模型无关，一致性得到验证。

结论：语义裁判是盲点质量的可用测量工具（方法贡献）；但 EpistemoBrain 的机制是整合级的（跨阶段塑造席位推理与召回），观测其收益需要强到能支撑完整协议不缺席的模型、或更轻的协议。这是工程后续项，不是静默假设。脚本化案例已隔离每个机制的结构贡献；本节记录真实模型下隔离失效的边界。

11 工程实现细节

11.1 技术栈

Python 3.12、FastAPI、Pydantic v2、SQLAlchemy 2.0 (async)、Alembic、PostgreSQL 16+、pgvector、Redis（首版不用，见第 3 章）、React + TypeScript + React Flow、Server-Sent Events、pytest。数据库是事实来源——即使任务状态也不只存在于内存或 workflow 框架内部。

11.2 模型网关

packages/models/openai_compatible.py 的 OpenAICompatibleModelGateway 对接任意 OpenAI-Chat-Completions 兼容端点 (DeepSeek / LongCat / 国内中转站)。特性:

- 流式 + 非流式回退: stream_invoke 解析 SSE 块, 失败回退到非流式 invoke, 任务不因流断而失败。
- thinking 模式: DeepSeek 的 reasoning_content 与 content 分开成 StreamEvent{kind, text}, 思维链原样捕获但标注为过程材料——只用于过程流展示, 不作为正式证据。
- Schema 修复: MAX_SCHEMA_REPAIR_ATTEMPTS=1。
- 总时限: INVOKE_TOTAL_TIMEOUT_SECONDS=300。
- 降级: DeepSeek 400 tool_choice 冲突自动降级。
- 成本审计: token/费用/延迟/重试/schema 状态全部落库 (models/audit.py)。

角色实现内没有散落厂商 SDK 调用——deliberation.py 只通过 ModelGateway 协议发 ModelRequest。

端点规范化与连通性探测 (models/endpoint_config.py): 一次真实事故设定了这条规则——研究者把 DeepSeek 控制台门户 (platform.deepseek.com) 当 API 端点填, 所有模型调用立即失败。因此用户输入从不原样存储: normalize_base_url 去掉空白和尾部斜杠、缺 scheme 补 https://、把已知控制台门户改写到对应 API 主机, 并以 hint 告知研究者。设置 API 拒绝保存未通过 probe_endpoint (单次、10s 超时的 chat-completions 连通测试) 的配置。API key 从不回显, 错误消息绝不引用它。

11.3 工具网关

packages/tools/http_gateway.py 的 HttpToolGateway 对接四家公开 REST API: OpenAlex、Crossref、Unpaywall、Semantic Scholar。OpenAlex/Crossref/Semantic Scholar 免凭证即可工作; Unpaywall 要求每次请求带联系邮箱 (POLISCOPE_TOOLS_CONTACT_EMAIL), 未设置时只有该供应商报错。工具调用同样经 AuditedToolGateway 落库 (延迟/费用/重试/错误)。

11.4 检索与取证管线

研究问题 -> 证据需求规划 -> 多源候选论文发现 -> 元数据归一化去重

-> 全文获取与结构化解析 -> 研究级/结果级证据抽取 -> 原文定位与引用绑定

-> 设计质量与适用边界审查 -> 证据独立性审计 -> 证据-主张蕴含验证

-> 写入 Scientific Event Ledger -> 通过 Evidence Gate -> 进入证据图

每一步留下事件记录, 因此可以回答: 为什么检索到这篇论文? 谁读取了它? 从哪一页提取了结论? 原文到底说了什么? 哪名科学家把它解释成支持某项主张? 审计员是否同意? 它与其他研究是否使用同一批数据?

PDF 解析用 `pymupdf`；**规范化作品**（Canonical Work）用 DOI → PubMed/arXiv ID → 标题 + 第一作者 + 年份 → 文本与作者模糊匹配的优先级识别预印本/正式版/扩展版为同一作品族，防止「三个版本被当作三项独立研究」。

11.5 报告组装

`packages/reports/service.py` 的 `ReportService` 从图、账本、任务行组装 `ResearchBrief`，两个格式（Markdown / JSON）来自同一次 build，不可能对议会结论产生分歧。REPORTING 阶段本身为空——报告由证据图驱动，而不是新一轮模型输出。

最终论文：`packages/reports/synthesis.py` 只使用账本已记录的材料（终审、条件化共识、被采纳来源与发现、局限）。模型失败时 `_fallback_integrated_paper` 从 brief + 终审判断确定性组装，绝不输出「尚未生成」空壳。论文是表达层，任务终态不因论文成功或失败而改变（失败记 `FINAL_PAPER_FAILED` 事件）。论文带 `standpoints / overall_conclusion / conclusion_evidence`（round-12 「整合结论详细化」）。

导出脱敏：所有导出过 `sanitize_export`，签名 URL 与本地路径被脱敏，避免上传材料通过报告泄露。

11.6 SSE 实时流：两条流一份契约

1. **账本流**（GET `/api/stream/{task_id}`）：`PHASE_STARTED` 等正式事件，驱动八阶段时间线。支持 `Last-Event-ID` 续传。
2. **过程流**（GET `/api/stream/{task_id}/process`）：`worker` 实时写入的易逝过程数据（席位开始思考、模型推理片段、输出 token、DOI 解析与文献检索命中）。重连客户端重读全量并按 `seq` 去重。

SSE 帧只有 `id:` 和 `data:`，没有 `event:` 行。原因是类型化帧只会送达注册了同名类型的监听器，客户端必须穷举事件词汇表并静默丢弃未知类型——审计轨迹曾因此只显示 53 个事件中的 37 个。事件类型放在 body 的 `kind` 字段里，客户端不可能收不到。

过程流数据层：`process_stream.py` 的 `ProcessStreamWriter` 缓冲批写（`flush_at=40`），flush 失败只降级为 warning、绝不破坏正在跑的 `run.process_stream` 表无外键（迁移 0016）——`worker` 认领任务时对任务行持有 `SELECT ... FOR UPDATE` 直到整轮议会事务提交，而 Postgres 外键检查会对父行取 `KEY SHARE` 锁，两边互相等待就是死锁（曾把整个 `worker` 冻结）。过程流是 best-effort 的，孤儿行无害。

11.7 权限即设计

PostgreSQL grants 实现写隔离：

应用角色（`poliscope_app`）：可追加账本、可读证据图，不能 `INSERT/UPDATE/TRUNCATE` 图
 投影器角色（`poliscope_projector`）：可写图、可推进事件状态，不能重写事件 `payload`
 两者都不能 `TRUNCATE`
 例外：应用角色拥有任务生命周期 `DELETE`（会话删除，迁移 0019）

这不是代码自律，是数据库事实——集成测试直接断言应用身份写图会被数据库拒绝。

11.8 部署

`docker-compose.yml` 六个服务：postgres、migrate（一次性跑 alembic + 建角色）、api、worker、web（nginx 静态 + 反代）、caddy（唯一对外端口）。Caddy 默认 :80 纯 HTTP；域名可用时改一行即自动签发 HTTPS 证书。GitHub Actions 部署：push main → SSH 登录 → `git pull -ff-only` + `docker compose up -d -build` + 落地页健康检查。服务器的 `.env` 不在 git 里、由部署者维护，模型 key 只进服务器 `.env`。

11.9 前端设计取向

界面像科研仪器与证据审查工具，不像驾驶舱：结论与局限并排、颜色只表示证据状态、被反驳节点淡化永不消失、缺口计数常驻页头且每个缺口被点名、无卡通 Agent 无霓虹无脉冲指示灯、完整支持 `prefers-reduced-motion`。产品中心是证据地图（React Flow），不是聊天框。

11.10 安全与伦理

系统输出不是临床诊断或医疗建议；涉及心理健康的问题自动附加科研辅助定位声明（`looks_like_mental_health` 匹配抑郁/焦虑/心理等词，只在相关领域附加）。不抓取或存储未授权的个人数据，不绕过付费墙。上传 PDF 的存储、日志和导出避免泄露用户材料。报告声明 AI 辅助、证据覆盖与系统局限。不以模型置信度替代统计不确定性或专家判断——这句话出现在每一份报告的局限一节，不是免责套话。